

Bsp: Fakultät von n

(Fakultät von negativen Zahlen und 0 wird als 1 definiert.)

$$n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$$

Bsp: $n=3$

$$i=3$$

$$res=1$$



$$i=2$$

$$res=3$$



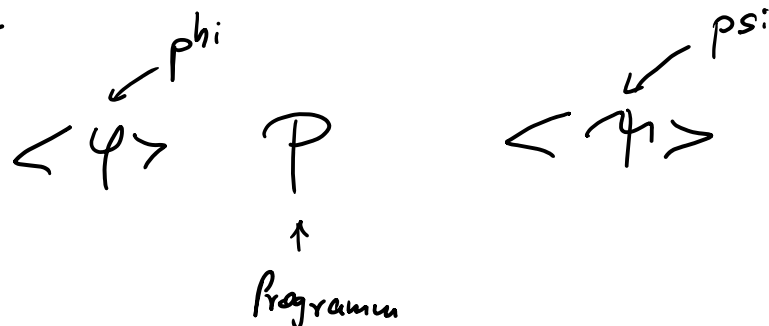
$$i=1$$

$$res=3 \cdot 2$$

Ergebnis: 6

Hoare - Kalkül

Tony Hoare

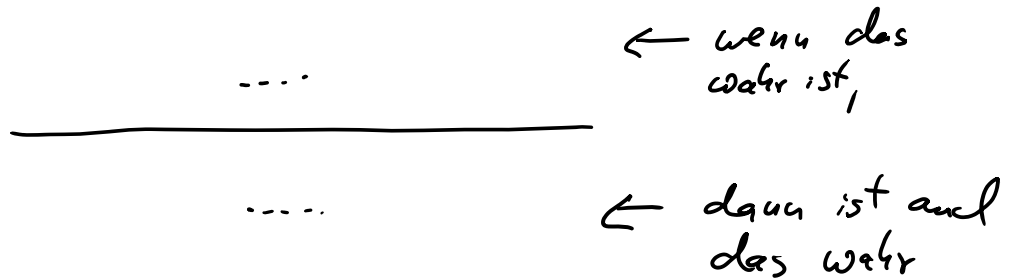


- Rahmen / Anleitung zur Verifikation

= 1/2 . 1/1 . 1. 0 01 . 1/1 . 1/1

- Namen / Anleitung zur Verifikation
- Verifikation leicht überprüfbar
- Verifikation teilweise automatisierbar

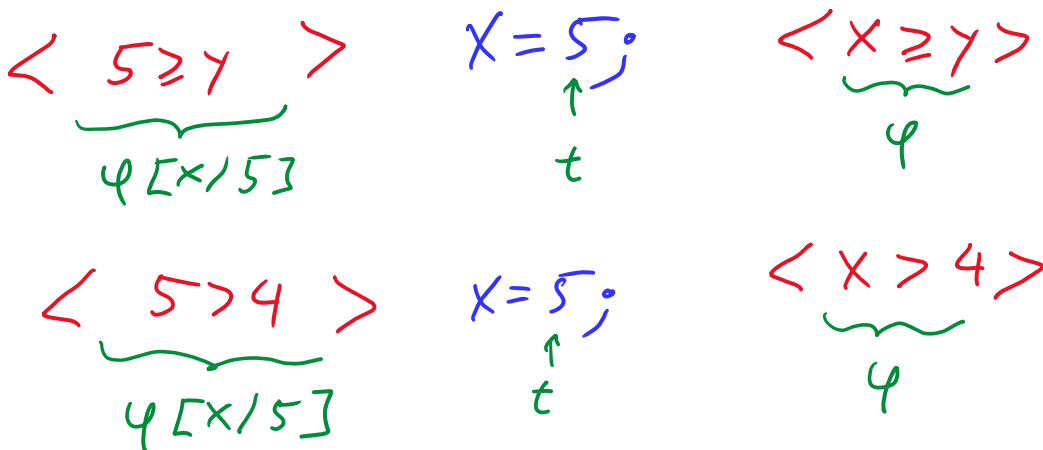
Schreibweise der Regeln:



Zuweisungsregel

- Oberhalb des Strichs steht nichts
 $\Rightarrow$ Aussage unterhalb des Strichs ist immer wahr
- Damit nach Ausführung von
 $x = t;$

die Nachbedingung φ gilt,
 muss vorher $\varphi[x/t]$ gegolten haben.



Schreibweise:

Schreibweise:

Ergänze Programmcode um Zusicherungen (Assertions), die wahre Aussagen über die Prog.-Variablen an dieser Stelle enthalten.

Konsequenzregel 1

Man kann die Vorbedingung verschärfen.

$$\begin{array}{ccc} \underbrace{\varphi}_{\text{true}} & & \underbrace{\varphi}_{\text{true}} \\ \langle 5 \geq Y \rangle & X = 5; & \langle X \geq Y \rangle \end{array} \left. \vphantom{\begin{array}{ccc} \underbrace{\varphi}_{\text{true}} & & \underbrace{\varphi}_{\text{true}} \\ \langle 5 \geq Y \rangle & X = 5; & \langle X \geq Y \rangle \end{array}} \right\} \begin{array}{ccc} \underbrace{\alpha}_{\text{true}} & & \underbrace{\varphi}_{\text{true}} \\ \langle 3 \geq Y \rangle & X = 5; & \langle X \geq Y \rangle \end{array}$$

$$\begin{array}{ccc} \underbrace{3 \geq Y}_{\alpha} & \Rightarrow & \underbrace{5 \geq Y}_{\varphi} \end{array}$$

Schreibweise:

- Wenn 2 Zusicherungen direkt untereinander stehen, dann muss die untere aus der oberen folgen.
- Wenn zwischen 2 Zusicherungen eine Anweisung steht, dann muss es einer Anwendung einer Regel des H-Kalküls entsprechen.

Konsequenzregel 2

Man kann die Nachbedingung abschwächen.

$$\begin{array}{ccc} \underbrace{\varphi}_{\text{true}} & & \underbrace{\varphi}_{\text{true}} \\ \langle 3 \geq Y \rangle & X = 5; & \langle X \geq Y \rangle \end{array} \left. \vphantom{\begin{array}{ccc} \underbrace{\varphi}_{\text{true}} & & \underbrace{\varphi}_{\text{true}} \\ \langle 3 \geq Y \rangle & X = 5; & \langle X \geq Y \rangle \end{array}} \right\} \begin{array}{ccc} \underbrace{\varphi}_{\text{true}} & & \underbrace{\beta}_{\text{true}} \\ \langle 3 \geq Y \rangle & X = 5; & \langle X+1 \geq Y \rangle \end{array}$$

$$\dots \vee \dots \quad X+1 \geq Y$$

$$\underbrace{x \geq y}_{\psi} \Rightarrow \underbrace{x+1 \geq y}_{\beta} \quad \left. \vphantom{\underbrace{x \geq y}_{\psi}} \right\} \langle 3 \geq 7 \rangle \quad x=5; \quad \langle x+1 \geq 7 \rangle$$

Logische Verknüpfungen:

$\wedge$	"und" ($\&\&$)	$\leftarrow b_1 \wedge b_2 = \text{true}$ falls $b_1 = \text{true}$ und $b_2 = \text{true}$
$\vee$	"oder" ($\ \ $)	
$\neg$	"nicht" (!)	$\leftarrow b_1 \vee b_2 = \text{false}$ falls sowohl $b_1 = \text{false}$ als auch $b_2 = \text{false}$

Sequenzregel

setzt Teilbeweise für Programme zusammen

$\langle \text{true} \rangle \leftarrow \psi$
 $\langle 5 = 5 \rangle$
 $x = 5;$
 $\langle x = 5 \rangle \leftarrow \psi$
 $\langle x * x + 6 = 31 \rangle$
 $res = x * x + 6;$
 $\langle res = 31 \rangle \leftarrow \beta$

Bedingungsregel 1

behandelt "if"-Anweisungen ohne "else"

$\langle \text{true} \rangle$

$\langle y = y \rangle$

| und:
 ψ

$\neg \beta$

$\langle true \rangle$

$\langle y = y \rangle$

$res = y;$
 $\langle res = y \rangle$

if $(x > y)$ {
 $\langle res = y \wedge x > y \rangle$
 $\langle x = \max(x, y) \rangle$
 $res = x;$
 $\langle res = \max(x, y) \rangle$
}

$\langle res = \max(x, y) \rangle$

und:

$\underbrace{\varphi} \quad \underbrace{\neg B}$
 $res = y \wedge \neg x > y \Rightarrow$

$res = \max(x, y)$
 $\underbrace{\quad}_{\varphi}$

Dies muss man zeigen,
sonst darf man
diese Zusicherung nicht
schreiben.

Bedingungsregel 2

$\langle true \rangle$

if $(x < 0)$ {
 $\langle true \wedge x < 0 \rangle$
 $\langle -x = |x| \rangle$
 $res = -x;$
 $\langle res = |x| \rangle$
}

else {
 $\langle true \wedge \neg x < 0 \rangle$
 $\langle x = |x| \rangle$
 $res = x;$
 $\langle res = |x| \rangle$
}

$\langle res = |x| \rangle$

Schleifenregel

Task: Finde eine Schleifeninvariante φ :

Idee: Finde eine Schleifeninvariante φ :

Wenn φ vor dem Schleifenrumpf P gilt
und der Schleifenrumpf wird ausgeführt
(weil die Bedingung B gilt),

dann gilt φ nach dem Schleifenrumpf immer noch.

Man muss eine Schleifeninvariante finden,
die 3 Eigenschaften erfüllt:

① φ muss eine Schleifeninvariante sein

$$\langle \varphi \wedge i > 1 \rangle$$

$$\text{res} = \text{res} * i; \quad i = i - 1;$$

$$\langle \varphi \rangle$$

② φ muss aus der Vorbedingung folgen

$$i = n \wedge \text{res} = 1 \quad \Rightarrow \quad \varphi$$

③ Nachbedingung muss aus der Schleifeninvariante φ
und der negierten Schleifenbedingung folgen:

$$\varphi \wedge \neg i > 1 \quad \Rightarrow \quad \text{res} = n!$$

Tipp: Starte mit der Nachbedingung und passe sie so
an, dass daraus eine Schleifeninvariante wird.

Führe dazu die Schleife mit typischen

Führe dazu die Schleife mit typischen Variablenwerten aus.

i	res	n
4	1	4
3	4	4
2	$4 \cdot 3$	4
1	$4 \cdot 3 \cdot 2$	4

Vorschlag:

$$i! \cdot \text{res} = n!$$

Zusicherungen können mit "assert" ins Java-Programm geschrieben werden.

Bei Ausführung mit `java -ea ...` werden Zusicherungen mit überprüft.

Terminierung

Finde Variante V , die ≥ 0 ist, wenn die Schleife ausgeführt wird und die in jedem Schleifendurchlauf kleiner wird.

Bsp: Variante ist i

- $i > 1 \Rightarrow i \geq 0$

- $$\langle i = n \wedge i > 1 \rangle$$

$$\langle i - 1 < n \rangle$$

$$\text{res} = \text{res} \cdot i$$

$\langle i-1 < m \rangle$

$yes = yes \vee i;$

$\langle i-1 < m \rangle$

$i = i-1;$

$\langle i < m \rangle$

Programmierer*in sollte sich zu jeder Schleife Invariante und Variante überlegen.

Additions-Bsp

Terminierung: Variante ist x

- $x > 0 \Rightarrow x \geq 0$

- $\langle x = m \wedge x > 0 \rangle$
 $\langle x-1 < m \rangle$
 $x = x-1;$

$$\langle x < m \rangle$$
$$yes = yes + 1;$$

$$\langle x < m \rangle$$

Subtraktions-Bsp

Terminierung: Variante ist $x-z$

- $x > z \Rightarrow x-z \geq 0$

$$\langle X - z = m \wedge X > z \rangle$$

$$\langle X - (z + 1) < m \rangle$$

$$z = z + 1;$$

$$\langle X - z < m \rangle$$

$$res = res + 1;$$

$$\langle X - z < m \rangle$$